

Claude Certified Architect

Foundations (CCAR-F) — Study Guide II: The Gap-Closer

The concrete artifacts, confusable-pair discrimination, scope matrix,
full glossary, anti-pattern catalog, worked math, and scope boundaries
that Guide I did not cover — built to push 689 past 720.

Use with Guide I. Guide I = *how to reason*. Guide II = *what to recognize & recall*.

Why This Second Guide Exists

A 689 means your reasoning is mostly sound but a handful of item types are leaking points. Those leaks are almost always one of four kinds: (1) you didn't *recognize* a concrete artifact (a config shape, a JSON field, a CLI flag); (2) you picked the wrong member of a *confusable pair* (fork vs resume, resource vs tool, **any** vs forced); (3) you missed a precise *term/definition* from the Appendix; or (4) you overthought an *out-of-scope* topic. This guide targets exactly those four. Study Domain 1 hardest — it is 27% of the exam, more than any other.

Domain weighting reminder (spend prep time proportionally): Agentic Architecture 27% · Claude Code 20% · Prompt/Structured Output 20% · Tool & MCP 18% · Context & Reliability 15%. Roughly 16, 12, 12, 11, and 9 of the 60 items.

Part 1 · Concrete Artifacts You Must Recognize on Sight

The exam shows you config and code and asks "what's wrong / where does this go / what does this guarantee." Recognition beats derivation under time pressure. These shapes are illustrative but faithful to what the guide names.

The agentic loop — the only correct control flow

```
# Loop is driven ONLY by stop_reason. No text parsing, no arbitrary cap.
messages = [user_msg]
while True:
    resp = claude.create(messages=messages, tools=tools)
    messages.append(resp) # append assistant turn
    if resp.stop_reason == "end_turn": # DONE - terminate
        break
    if resp.stop_reason == "tool_use": # CONTINUE
        results = [execute(tc) for tc in resp.tool_calls]
        messages.append(tool_result_msg(results)) # feed results back
```

What the exam will show as "wrong": if "done" in resp.text: break (parsing NL) · if i > 10: break as the *primary* stop · checking for assistant text as a completion signal. All three are named anti-patterns.

MCP server config — project vs user scope, env-var expansion

```
// .mcp.json (project scope - committed, shared with the whole team)
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" } // expansion, no secret committed
    }
  }
}
// ~/.claude.json (user scope - personal / experimental, NOT shared)
```

Env-var expansion (`${GITHUB_TOKEN}`) is the mechanism for keeping credentials out of version control. Tools from **all** configured servers are discovered at connection time and usable simultaneously.

Structured MCP error response (not "Operation failed")

```
{
  "isError": true,
  "errorCategory": "transient", // transient | validation | business | permission
  "isRetryable": true,
  "description": "Order service timed out after 5s"
}
// business rule violation example:
{ "isError": true, "errorCategory": "business",
  "retryable": false, "description": "Refund exceeds $500 policy limit" }
```

Structured output via tool_use + JSON schema (nullable / enum+other)

```
{
  "name": "extract_invoice",
  "input_schema": {
    "type": "object",
    "properties": {
      "vendor": { "type": ["string", "null"] }, // nullable -> no fabrication
      "category": { "type": "string",
        "enum": ["goods", "services", "other", "unclear"] },
      "category_detail": { "type": ["string", "null"] }, // used when "other"
      "stated_total": { "type": "number" },
      "calculated_total": { "type": "number" } // flag mismatch = semantic check
    },
    "required": ["stated_total"] // keep required minimal
  }
}
```

Guarantee vs no-guarantee: tool_use + strict schema eliminates JSON *syntax* errors. It does NOT stop *semantic* errors (totals not summing, value in wrong field). Catch those with paired fields (`stated_total` vs `calculated_total`) and `conflict_detected` booleans.

tool_choice — three shapes

```
"tool_choice": {"type": "auto"} // may return text OR call a tool
"tool_choice": {"type": "any"} // MUST call some tool (model picks which)
"tool_choice": {"type": "tool", "name": "extract_metadata"} // MUST call THIS one
```

Path-scoped rule file (.claude/rules/)

```
# .claude/rules/testing.md
---
paths: ["**/*.test.tsx", "**/*.test.ts"] # loads ONLY when editing matches
---
Use React Testing Library. One assertion per test. Mock network via msw.
```

SKILL.md frontmatter (the three tested keys)

```
---
name: codebase-audit
context: fork # run in isolated sub-agent → don't pollute main chat
allowed-tools: [Read, Grep, Glob] # restrict tools during execution
argument-hint: "<directory to audit>" # prompt dev if invoked w/o args
---
```

Claude Code in CI (the exact flags)

```
claude -p "Review this PR for security issues" \
  --output-format json --json-schema ./finding.schema.json
# -p / --print = non-interactive (prevents hang). NOT CLAUDE_HEADLESS, NOT --batch.
```

Persisted "case facts" block (survives summarization)

```
# Injected into EVERY prompt, OUTSIDE the summarized history:
CASE FACTS
  order_id: 48213   refund_amount: $129.99   status: shipped_damaged
  customer_id: C-7781 (verified)   promised_by_agent: full refund + reship
  key_dates: ordered 2026-06-14, delivered 2026-06-20
```

Structured claim-source mapping (preserved through synthesis)

```
{ "claim": "AI reduced illustration turnaround 40%",
  "evidence": "...survey of 200 studios...",
  "source": "https://example.org/report",
  "pub_date": "2026-03", // prevents temporal false-conflicts
  "relevance": 0.9 }
```

Parallel subagent spawning (one turn, multiple Task calls)

```
# Coordinator emits MULTIPLE Task tool calls in a SINGLE response → parallel.
# allowedTools MUST include "Task". Each subagent gets its context IN its prompt.
Task(agent="web_search", prompt="Music industry AI impact. Return claim-source JSON.")
Task(agent="web_search", prompt="Film production AI impact. Return claim-source JSON.")
Task(agent="doc_analysis", prompt="Analyze attached papers. Include pub dates.")
```

Part 2 · Confusable Pairs (where borderline points hide)

The exam loves questions where two options are both "reasonable" and you must pick the correct member of a pair. Drill these until the discrimination is automatic.

If the scenario says...	It's THIS, not that
Prior context mostly still valid, continue a named investigation	--resume <session> — not fork, not fresh start
Explore two divergent approaches from one shared baseline	fork_session — not resume (that's linear continuation)
Prior tool results are stale after code changed a lot	Start fresh with an injected structured summary — not resume (stale results mislead)
Read-only content/catalog exposed to the agent (schemas, issue lists, doc trees)	MCP resource — not a tool
An action the agent performs (refund, search, write)	MCP tool — not a resource
Must call some tool, don't care which	tool_choice "any" — not forced
A specific tool must run first (e.g. metadata before enrichment)	Forced tool_choice {type:tool,name:...} — not "any"
Guaranteed compliance with a business/financial/identity rule	Hook / prerequisite gate — not a system-prompt instruction
Nudge behavior where occasional misses are tolerable	Prompt guidance / few-shot — hooks would be overkill
Transform tool results before the model reads them (normalize timestamps)	PostToolUse hook — not tool-call interception
Block an outgoing tool call that violates policy	Tool-call interception hook — not PostToolUse
Search file contents for a pattern (callers, error strings)	Grep — not Glob
Find files by name/extension pattern	Glob — not Grep
Edit failed because the anchor text isn't unique	Read + Write fallback — not retrying Edit
Predictable multi-aspect steps known in advance	Prompt chaining (fixed sequence) — not dynamic decomposition
Subtasks depend on what you discover as you go	Dynamic/adaptive decomposition — not a fixed pipeline
Convention for a file type scattered across many directories	.claude/rules/ with glob — not a subdirectory CLAUDE.md
Convention for everything under one directory tree	Directory-level CLAUDE.md is acceptable there
Always-on universal standards	CLAUDE.md — not a skill
On-demand, task-specific workflow, invoked when needed	Skill — not CLAUDE.md
Team-wide, arrives on clone/pull	Project scope (.claude/ , .mcp.json) — not ~/.claude
Personal/experimental, must not affect teammates	User scope (~/.claude/ , ~/.claude.json)
Large-scale / architectural / multiple approaches / many files	Plan mode — not direct execution
Single well-scoped change, clear stack trace	Direct execution — not plan mode
Retry will fix it (format mismatch, structural output error)	Retry-with-error-feedback works
Info is simply absent from the source document	Retry is futile — route to human / mark null
JSON braces/quotes malformed	Syntax error → eliminated by tool_use schema
Totals don't sum / value in wrong field	Semantic error → needs validation logic, not schema
Query succeeded but found nothing	Valid empty result — not an access failure
Query couldn't run (timeout, service down)	Access failure — needs a retry decision
Customer explicitly asks for a human	Escalate immediately — don't investigate first
Customer frustrated but issue is within capability	Acknowledge + offer resolution; escalate only if they reiterate
Policy is silent/ambiguous on the request	Escalate (policy gap) — not autonomous guessing
Non-blocking, latency-tolerant, cost-sensitive	Message Batches API
Blocking, developer waiting, pre-merge	Synchronous API — never batch
Reviewing code the same session generated	Independent review instance — not self-review / more thinking
Isolate verbose discovery output during planning	Explore subagent
Isolate a verbose skill's output	context: fork in SKILL.md
Shrink an already-full context mid-session	/compact
Tool misrouting between similar tools	Rewrite descriptions (root cause) — few-shot is second-line
Inconsistent format / ambiguous-case judgment	Few-shot examples — descriptions won't fix judgment

Part 3 · Master Scope Matrix (Domain 3 is 20% — memorize this)

Every "where does this go?" question reduces to this table. Project = version-controlled, shared with the team on clone/pull. User = personal, never shared.

Artifact	User scope (personal)	Project scope (team, VCS)	Notes
CLAUDE.md	~/.claude/CLAUDE.md	.claude/CLAUDE.md or root CLAUDE.md ; also subdirectory CLAUDE.md	Directory-level applies only to that tree
Slash commands	~/.claude/commands/	.claude/commands/	One file per command
Skills	~/.claude/skills/ (use a different name)	.claude/skills/ (SKILL.md)	Personal variant name avoids affecting teammates
Rules	—	.claude/rules/*.md with paths: frontmatter	Glob-scoped, load on matching file edits
MCP servers	~/.claude.json	.mcp.json	Use \${VAR} expansion for secrets

Diagnostic pattern (appears verbatim on exam): "New teammate isn't getting an instruction." → It's in *user-level* config; move it to *project-level*. Verify loaded memory files with /memory . Keep CLAUDE.md modular with @import ; split monoliths into .claude/rules/ .

Part 4 · The Six Scenarios as Reference Architectures

You get 4 of these 6. For each: the correct design and the trap the exam plants.

Scenario 1 — Customer Support Resolution Agent (target: 80%+ first-contact)

MCP tools: get_customer , lookup_order , process_refund , escalate_to_human . Correct design: **prerequisite gate** forcing verified customer before order/refund ops; **tool-call interception hook** to block refunds over threshold; **explicit escalation criteria + few-shot; structured handoff** on escalation; persistent **case-facts block**. Traps: prompt-based "mandatory" verification; self-confidence or sentiment escalation; heuristic pick on multiple customer matches (ask for another identifier instead).

Scenario 2 — Code Generation with Claude Code

Correct: custom / commands in .claude/commands/ ; CLAUDE.md standards; **plan mode for architectural/multi-file, direct for scoped fixes**; concrete input/output examples for tricky transforms. Trap: deferring plan mode "until complexity emerges" when it's already stated.

Scenario 3 — Multi-Agent Research System

Coordinator delegates to search / analysis / synthesis / report subagents. Correct: allowedTools includes Task ; explicit context in each prompt; parallel Task calls; structured claim-source + pub-date output; **structured error context** on subagent failure; coverage-gap annotation; scoped verify_fact for synthesis. Traps: blaming downstream agents for a narrow coordinator decomposition; giving synthesis all web tools; suppressing or fatal-failing on one subagent error; arbitrarily picking one of two conflicting stats.

Scenario 4 — Developer Productivity with Claude

Built-ins Read/Write/Bash/Grep/Glob + MCP. Correct: incremental exploration (Grep entry points → Read to trace); subagents/scratchpad for verbose exploration; scoped tools per role; enrich MCP tool descriptions so capable MCP tools aren't skipped for built-in Grep. Trap: reading all files upfront; too many tools degrading selection.

Scenario 5 — Claude Code for CI/CD

Correct: -p for non-interactive; --output-format json --json-schema for PR comments; CLAUDE.md supplies review criteria/fixtures; independent review instance (not self-review); include prior findings to avoid duplicate comments; batch overnight jobs, sync for pre-merge; per-file + integration passes for big PRs; explicit criteria to cut false positives. Traps: fake flags; batch for blocking checks; bigger context to fix diluted reviews.

Scenario 6 — Structured Data Extraction

Correct: tool_use + JSON schema; nullable fields; enum with "other"+detail and "unclear"; validation-retry with error feedback (but not when info is absent); few-shot for varied doc formats; batch for volume with custom_id resubmission; field-level confidence + stratified sampling + accuracy by doc-type/field before automating. Traps: required fields forcing fabrication; retrying when the data isn't in the source; trusting 97% aggregate accuracy without segmenting.

Part 5 · Complete Anti-Pattern Catalog

"Which of these is an anti-pattern / what should you avoid" is a recurring frame. If an option matches one of these, it's almost certainly the wrong choice (or the right answer to a "what's wrong" question).

Domain	Anti-pattern
Agentic loop / orchestration	Parsing natural-language text to decide loop termination
	Arbitrary iteration cap as the primary stop mechanism
	Checking assistant text content as a completion signal
	Overly narrow coordinator decomposition → missing whole subtopics
Tools / MCP	Uniform "Operation failed" error responses
	Overloading an agent with 18 tools instead of 4-5
	Giving an agent tools outside its specialization
Errors / reliability	Silently suppressing errors (empty result marked success)
	Terminating the whole workflow on a single subagent failure
Escalation	Sentiment analysis as a complexity/escalation proxy
	LLM self-reported confidence to route hard cases
	Heuristic selection when multiple records match (should ask)
Prompting / output	Vague instructions ("be accurate", "be conservative") for precision
	Required fields forcing the model to fabricate missing values
	Retrying extraction when the info is absent from the source
Review / context	Larger context window to fix attention dilution
	Self-review in the same session that generated the code
	Progressive summarization that flattens numbers/dates/promises
Config	Team instruction placed in user-level config
	Subdirectory CLAUDE.md for a file type scattered across dirs
	Referencing non-existent features (CLAUDE_HEADLESS, --batch)
Batch API	Using batch for blocking/pre-merge workflows (no SLA)

Part 6 · Worked Numbers (batch, SLA, and thresholds)

SLA math (the exam does this). Batch processing can take up to **24h**. If you must guarantee a **30h** end-to-end SLA, submit batches on a cadence of at most **30 – 24 = 6h** windows — commonly stated as "submit every 4 hours to comfortably guarantee 30h." The logic: worst-case wait = submission-interval + processing-window, so interval must be ≤ SLA – max-processing-time.

- **Batch cost/latency:** 50% cheaper, up to 24h, no guaranteed SLA, `custom_id` correlates pairs, no multi-turn tool calling in one request.
- **Batch failure handling:** resubmit only failed `custom_id`s; chunk documents that exceeded context; refine prompt on a sample first to raise first-pass success.
- **Support target:** 80%+ first-contact resolution; the 55%-too-low case is fixed by escalation-calibration criteria, not infrastructure.
- **Tool count:** 4-5 good, ~18 degrades selection.
- **Few-shot:** 2-4 targeted examples.
- **Refund gate example:** block > \$500 via hook, redirect to escalation.
- **Accuracy trap:** 97% aggregate can hide bad performance on a doc type/field → segment before automating.

Part 7 · Glossary of Every Appendix Term

A recall gap on any of these can cost an item. One line each.

- Agentic loop** — repeat: call model, check `stop_reason`, run tools, append results, until `end_turn`.
- stop_reason** — `"tool_use"` = continue and run tools; `"end_turn"` = finished.
- AgentDefinition** — config for a subagent: description, system prompt, tool restrictions.
- Task tool** — spawns subagents; coordinator's `allowedTools` must include `"Task"`.
- allowedTools** — the list of tools an agent may call; gates subagent spawning and scoping.
- PostToolUse hook** — intercepts tool *results* to transform/normalize before the model sees them.
- Tool-call interception hook** — intercepts outgoing tool *calls* to enforce policy (block/redirect).
- Hooks vs prompts** — hooks = deterministic guarantees; prompts = probabilistic guidance.
- MCP** — Model Context Protocol; standard for exposing tools and resources to the agent.
- MCP tool** — an action the agent invokes; **MCP resource** — read-only content/catalog exposed to reduce exploratory calls.
- isError flag** — MCP field signaling a tool failure back to the agent.
- errorCategory** — transient / validation / business / permission; drives recovery.
- isRetryable / retrieable** — whether the agent should retry; false for business/permission errors.
- .mcp.json** — project-scoped MCP config (shared); **~/.claude.json** — user-scoped (personal).
- Env-var expansion** — `${TOKEN}` in config to keep secrets out of VCS.
- CLAUDE.md hierarchy** — user (`~/.claude`) / project (`.claude` or `root`) / directory; always-loaded standards.
- @import** — pull external files into CLAUDE.md for modularity.

.claude/rules/ — topic rule files; YAML `paths:` globs for conditional loading.

.claude/commands/ — project slash commands (shared); **~/.claude/commands/** — personal.

.claude/skills/ + SKILL.md — on-demand workflows; frontmatter: `context: fork` , `allowed-tools` , `argument-hint` .

context: fork — run a skill in isolated sub-agent context so output doesn't pollute the main chat.

argument-hint — frontmatter prompting for required params when a skill is invoked bare.

Plan mode — explore/design before changing; for complex, architectural, multi-file, multi-approach work.

Direct execution — make changes immediately; for simple, well-scoped tasks.

Explore subagent — isolates verbose discovery and returns a summary to preserve main context.

/memory — shows which memory/CLAUDE.md files are loaded (diagnose inconsistency).

/compact — reduce context usage mid-session when it fills with verbose output.

--resume <name> — continue a specific named prior session.

fork_session — branch from a shared baseline to explore divergent approaches in parallel.

Named sessions — labeled sessions you can resume later.

Session context isolation — an independent instance reviews better than the generating session.

-p / --print — non-interactive CLI mode for CI (prevents input hangs).

--output-format json / --json-schema — structured CLI output for automated PR posting.

tool_use + JSON schema — reliable schema-compliant output; kills syntax errors, not semantic ones.

tool_choice — auto (may text) / any (must call some tool) / forced (must call named tool).

max_tokens — output length cap on an API request.

Message Batches API — 50% cost, ≤24h, no SLA, `custom_id` , no multi-turn tool calling.

custom_id — correlates batch requests to responses; used to resubmit only failures.

JSON Schema features — required vs optional, enum, nullable, "other"+detail, strict mode.

Pydantic — validates output; distinguishes semantic validation errors from schema syntax errors.

Validation-retry loop — resend document + failed extraction + specific error for self-correction.

Built-in tools — Read, Write, Edit (unique-match), Bash, Grep (contents), Glob (filenames).

Few-shot prompting — 2-4 examples for ambiguous cases, format, and generalization.

Prompt chaining — fixed sequential decomposition into focused passes.

Dynamic decomposition — subtasks generated adaptively from intermediate findings.

Lost-in-the-middle — models drop middle content; put key findings first, use headers.

Progressive summarization — risk of flattening numbers/dates/promises; keep a case-facts block.

Scratchpad files — persist key findings across context boundaries in long exploration.

Manifest — exported agent state a coordinator loads on resume for crash recovery.

Field-level confidence — per-field score; calibrate with a labeled validation set to route review.

Calibration — align stated confidence with actual accuracy using labeled data.

Stratified sampling — sample within strata (doc type/field) to measure true error rates.

detected_pattern — field tagging what triggered a finding, to analyze false-positive patterns.

conflict_detected — boolean flagging inconsistent source data.

Claim-source mapping — structured claim + evidence + source + date preserved through synthesis.

Coverage annotation — marks which findings are well-supported vs gaps from unavailable sources.

Part 8 · Out-of-Scope — Do NOT Study or Overthink These

Half of exam mistakes at your level are choosing a technically-fancy answer that lives outside the tested scope. If an option is about one of these, it's a distractor.

- Fine-tuning or training custom models
- API authentication, billing, account management, API-key rotation, OAuth, auth protocols
- Deep implementation of specific languages/frameworks (beyond tool + schema config)
- Deploying/hosting MCP servers (infra, networking, containers)
- Claude's internal architecture, training process, model weights
- Constitutional AI, RLHF, safety-training methodology
- Embedding models / vector database implementation
- Computer use (browser/desktop automation)
- Vision / image analysis
- Streaming API / server-sent events
- Rate limiting, quotas, pricing calculations
- Specific cloud provider config (AWS/GCP/Azure)
- Performance benchmarking / model comparison metrics
- Prompt caching *implementation* (know it exists; nothing deeper)
- Token counting algorithms / tokenization internals

Exam reflex: an answer proposing a trained classifier, a vector DB, a fine-tune, sentiment analysis, or streaming is nearly always the wrong, out-of-scope, over-engineered distractor. The in-scope right answer is usually a description fix, an explicit criterion, a few-shot set, a hook/gate, a scope change, or a structured data format.

Part 9 · Exam Mechanics & Time Strategy

- **60 items, 120 minutes** = 2 min/item average. Multiple-choice and multiple-response; each item states how many to select — read that count carefully on multi-response items.
- **4 scenarios drawn from 6.** Each scenario frames several items; read the whole stem first — the decisive detail is often buried in the "logs show..." sentence.
- **Scoring:** scaled 100–1000, cut score 720, criterion-referenced (you're judged against a standard, not a curve). Score report gives percent-correct by domain, but pass/fail is total scaled score only.
- **Multi-response items:** partial guesses hurt; only select the exact number requested and be sure of each.
- **Retake:** waits are 14 / 30 / 90 days after 1st/2nd/3rd fail; up to 4 attempts per rolling 12 months; fee each time.
- **Time tactic:** answer the reflexive ones fast (scope, CLI flags, stop_reason), bank time for the multi-step "most effective" judgment items where distractors are close.

Part 10 · Final 40-Fact Rapid Recall

1. Loop stops only on `end_turn` ; continues on `tool_use` .
2. Never parse text or use an iteration cap to end the loop.
3. Tool results are appended to history each iteration.
4. Coordinator owns all inter-agent communication (hub-and-spoke).
5. Subagents don't inherit context — put it in the prompt.
6. Spawn subagents with the Task tool; allowedTools must include "Task".
7. Parallel = multiple Task calls in one response.
8. Missing subtopic coverage = coordinator decomposed too narrowly.
9. Deterministic requirement → hook/gate, never a prompt.
10. PostToolUse hook normalizes results; interception hook blocks calls.
11. Prompt chaining = fixed steps; dynamic = adaptive from findings.
12. resume = continue; fork_session = branch; fresh+summary when results are stale.
13. Tool descriptions are the primary selection mechanism — fix them first.
14. Structured errors: errorCategory, isRetryable, description.
15. Access failure ≠ valid empty result.
16. Too many tools degrades selection (18 vs 4-5).
17. Scoped verify_fact for synthesis; not all web tools.
18. tool_choice: auto / any / forced-by-name.
19. .mcp.json = project; ~/.claude.json = user; \${VAR} for secrets.
20. MCP resource = content catalog; MCP tool = action.
21. Grep = contents; Glob = filenames; Edit fails → Read+Write.
22. CLAUDE.md hierarchy: user / project / directory; @import for modularity.
23. Team instruction missing → it's in user-level, move to project.
24. .claude/commands/ shared; ~/.claude/commands/ personal.
25. SKILL.md: context: fork, allowed-tools, argument-hint.
26. .claude/rules/ + glob paths for file types across directories.
27. Plan mode = complex/architectural/multi-file; direct = scoped.
28. Don't defer plan mode when complexity is already stated.
29. Concrete I/O examples beat prose for transformations.
30. CI: -p (--print); --output-format json + --json-schema.
31. Independent instance reviews better than self-review.
32. Explicit categorical criteria beat "be conservative".
33. Few-shot: 2-4 examples for ambiguity/format/generalization.
34. tool_use schema kills syntax errors, not semantic ones.
35. Nullable fields prevent fabrication; enum "other"/"unclear" for edges.
36. Retry fixes format; useless when info is absent from source.
37. Batch: 50%, ≤24h, no SLA, custom_id, no multi-turn tools; overnight only.
38. Attention dilution → per-file + integration passes, not bigger context.
39. Case-facts block + findings-first ordering beat lost-in-the-middle.
40. Segment accuracy by doc-type/field before automating (97% can lie).

Your specific path from 689 to 720. You don't need new knowledge breadth — you need to stop losing the close-call items. Do this: (1) drill Part 2 confusable pairs until instant; (2) memorize Part 3 scope matrix cold; (3) internalize that fancy answers (classifiers, sentiment, vector DBs, bigger models, self-confidence) are traps (Part 8); (4) on every "most effective / first step" item, pick the proportionate root-cause fix. That's roughly where 31 points live. You've got this.